
A tour of the niri scrolling-tiling Wayland compositor

Published by

Joe Brockmeier

Estimated reading time

13 min

Added on

2026-06-23

Address

<https://lwn.net/Articles/1025866/>

A tour of the niri scrolling-tiling Wayland compositor

Did you know...?

LWN.net is a subscriber-supported publication; we rely on subscribers to keep the entire operation going. Please help out by [buying a subscription](#) and keeping LWN on the net.

[Niri](#) is a relatively new Rust-based [compositor](#) for Wayland with a different take on [tiling window management](#): windows are placed onscreen in an "infinite" row that can expand beyond the bounds of the visible workspace. It is not a full-blown desktop environment, but niri may be a suitable option for Linux users who want tiling features and the minimalism of a window manager for Wayland.

Scrollable tiling

Floating window management is the norm for Linux desktop environments (as well as macOS and Windows), but tiling window management has been around for a long time, arguably as far back as [Xerox Star](#) systems in the early 1980s. There are plenty of window managers and compositors that offer tiling for Linux users: [awesome](#), [i3](#), [ratpoison](#), and [sway](#), to name just a few.

A bit more than 13 years ago, Jesse McClure [announced](#) a slightly different approach to tiling window management with the [ScrollWM](#) project. It provided a single large virtual desktop "through which one could scroll smoothly". Since then, there have been a number of other scrollable tiling implementations, including [PaperWM](#), an extension that enables scrollable tiling of windows for GNOME, which LWN [looked at](#) in January.

Inspired by PaperWM, Ivan Molodetskikh started development on the niri compositor in 2023. According to the niri project's README, Molodetskikh liked PaperWM's model for window management, but he decided to write his own compositor to overcome limitations in PaperWM's ability to isolate workspaces to individual monitors. With niri, each display has its own, discrete workspace that does not overlap with others.

He jump-started niri development by using [Smithay](#), a Rust-based project that is designed to speed up compositor development. It provides "objects and interfaces implementing common functionalities that pretty much any compositor will need, in a generic fashion". It seems to be a fairly popular starting point for Wayland compositor development and is used by [several other projects](#), including the new [COSMIC](#) desktop environment, which we [looked at](#) in August 2024. Smithay is available under the MIT license, while niri is licensed under the GPLv3.

Getting niri

Niri is available in some, but not all, major Linux distributions. The [getting started](#) page lists distributions that do have niri, which includes recent versions of Alpine, Arch, Fedora, Gentoo, openSUSE Tumbleweed, and others. Currently, it is not available in any of the Debian repositories or for Ubuntu. Instructions for building niri from source are also provided, of course. Prospective users may want to take note of guidance for [using niri with NVIDIA drivers](#) or [trying to run it on Arm systems](#), including Asahi Linux.

At the moment, the niri project is mostly a one-person show; it has had small contributions from more than 100 people, but only three developers have landed more than ten commits to the project, while Molodetskikh has more than 1,900 since August 2023. At the beginning of 2025, he [declared](#) that niri was now "sufficiently featureful to graduate from v0.1" and released version 25.01. The project now uses a "year.month" versioning scheme, but it has no set release schedule; new versions are released whenever he feels that it's a good time for a new version.

The most recent major release of niri is [v25.05](#), announced on May 17. It brought a few major changes, including an [overview feature](#) that lets users see all open windows and workspaces, and [features to black out specific windows](#) when screencasting. A minor "hotfix" release, [25.05.1](#), was announced on May 25.

I installed niri 25.05.1 on a laptop with Intel graphics running Fedora 42 Workstation. This added niri as an option under the GNOME Display Manager (GDM), as well as configuring [portals](#) and systemd services for niri. It also installed and set up a few helper programs, such as [fuzzel](#), [mako](#), and [Waybar](#), to provide additional functionality. Fuzzel is an application launcher for Wayland, mako implements the freedesktop.org [desktop notifications specification](#), and Waybar is a highly customizable panel application for Wayland similar to the [GNOME](#) and [KDE Plasma](#) panels. Users installing niri from source may want to read the [example systemd setup](#) documentation for guidance.

Users are on their own to pick and choose other applications that typically come with a desktop environment: a file manager, terminal emulator, media player, package management utility, image viewer, and the rest.

Using niri

The first thing a user will see when logging into niri is an "Important Hotkeys" dialog that lists a handful of the most commonly used movement and window-management keys. The dialog is dismissed as soon as the user presses a key, clicks a mouse button, or similar. It can be summoned again by using the Super+Shift+`/` key combination. The Super key is usually the Windows key on modern PC keyboards; niri uses this as its standard modifier key (or Mod), though it can be [changed](#) to another modifier key if necessary.

Niri's default configuration also launches the Waybar panel. It has its own [configuration](#), `~/.waybar/config.jsonc` on Fedora, to define its size, position, and the [modules](#) it displays, as well as how they operate.

Waybar has a ton of modules to add functionality, and many options that users can configure. The project has several [examples](#) of configuring Waybar to mimic the macOS panel and Dock, the Windows 10 menu, and some extremely minimal configurations. In the interest of not being lost down a customization rabbit hole, so far I have avoided making more than a few light changes to Waybar—with one exception.

Generally, I would expect a desktop panel to have a menu for suspending, rebooting, or powering off a system—but that was not available by default with Waybar on Fedora, so I added one to the `config.jsonrc`. This requires two changes to that configuration file—adding a custom module and then placing the module on the Waybar. The module definition looks like this:

```
"custom/power": {  
    "format" : " ? ",
```

```
"tooltip": true,  
"menu": "on-click",  
"menu-file": "$HOME/waybar/power_menu.xml",  
"menu-actions": {  
    "shutdown": "shutdown",  
    "reboot": "reboot",  
    "suspend": "systemctl suspend",  
    "hibernate": "systemctl hibernate"  
}  
}
```

A sample power_menu.xml configuration is available [on GitHub](#).

Waybar modules can be placed on the right, left, or in the center of the panel. I added the power button to the left-hand side of the panel, like so:

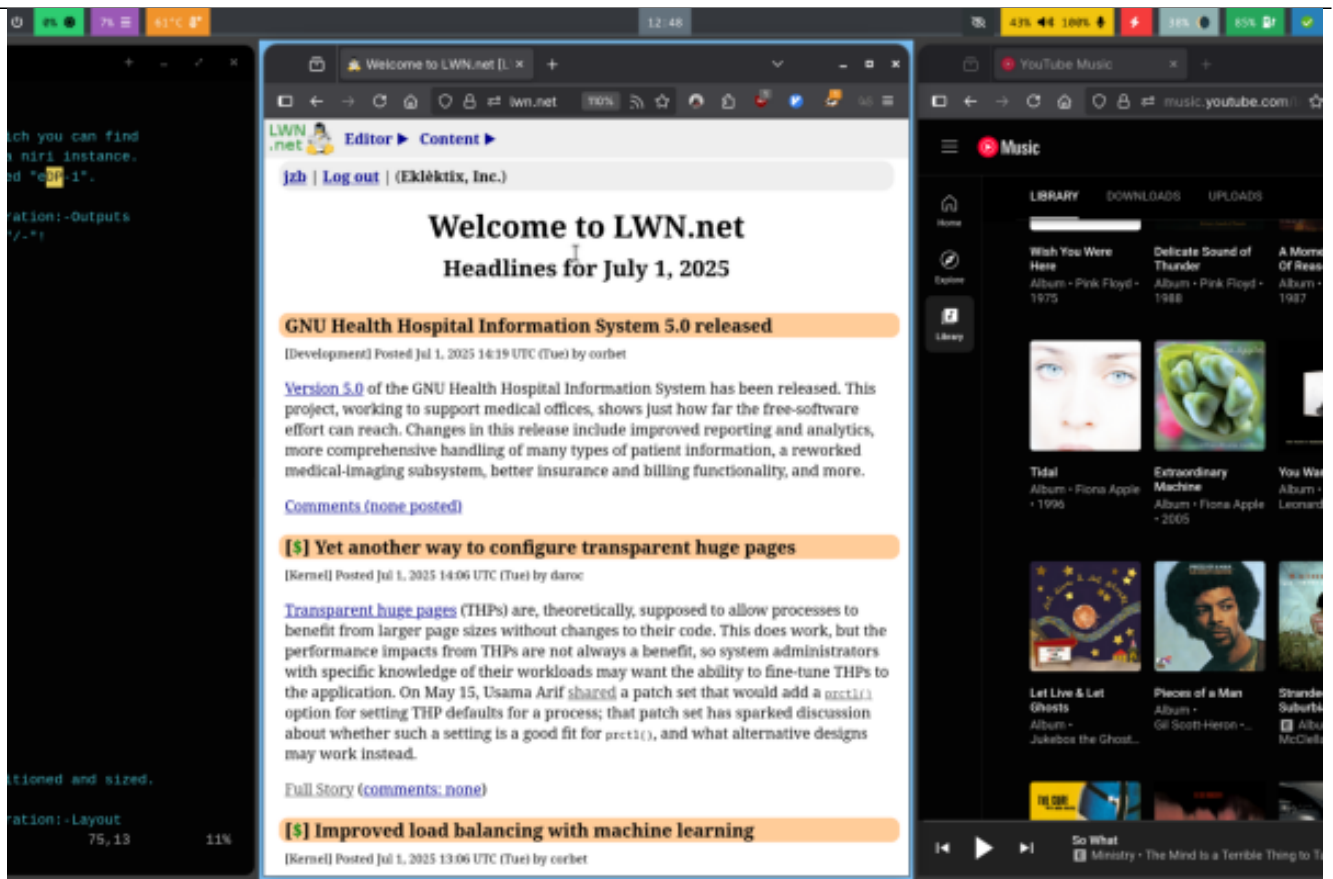
```
"modules-left": [  
    "custom/power"  
],
```

Note that Waybar [does not automatically pick up changes](#) to its configuration, so it will need to be restarted before any modifications are available.

Like most tiling window managers, niri is meant to be keyboard-driven; all of the operations to move and resize windows can be performed without the mouse. However, it is not as rigid as some (like ratpoison) that only allow users to manipulate windows with the keyboard—it is possible to move windows around and resize them with the mouse if that's more comfortable.

Assuming that the fuzzel application launcher is installed with niri, Super+D will bring up its dialog to launch applications. It will display a list of possible options, such as Firefox, which can be selected using the arrow key, or one can start typing a program name and fuzzel will attempt to find matching programs.

Once a program is launched, niri will place its window in a column on the display at a preset width and height. The default height is 100% of the display, and the widths are one-third, one-half, and two-thirds of the display's width. Users can toggle a window's width with Super+R; this will cycle the window through the available preset widths. Shift+Super+R will cycle through the available presets for height, either 100% or 50% of the display by default.



Each new window will be placed to the right of the window that currently has focus. If there are more windows than space on the display's workspace, niri will "scroll" older windows off the screen to the left. Niri also has dynamic workspaces for each display, which are arranged vertically. It has keybindings to move the focus to different windows, move windows between workspaces and displays, and so forth. For example, Super+H will select the window to the left of the active window, Super+Ctrl plus the arrow keys will move the window left or right, respectively, and Super+C will center the active window on the display and adjust the rest of the windows accordingly.

Users can perform window management with a mouse or touchpad as well, so niri is suitable for people who prefer that method, too.

Advanced window placement

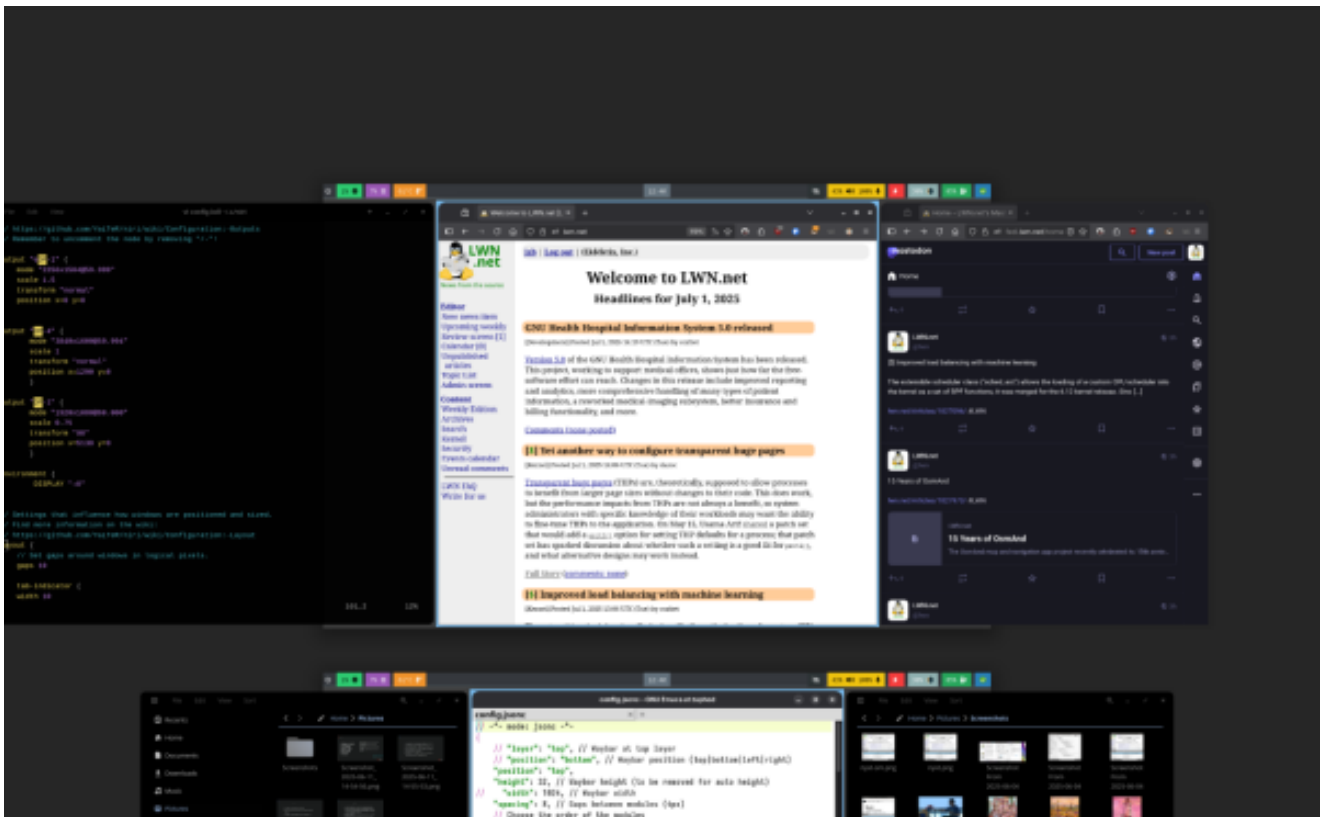
There are times when one might want to have a [floating window](#) that shows on top of the tiled windows; Super+V will pop a window out of the tiling layer and display it above the rest of the windows. Hitting Super+V again will return a window to the tiling layer. Niri does this automatically for things like dialog boxes, file choosers, and other child windows.

Windows can be added to a [tab group](#) as well. This will group multiple windows together in one column with a tab interface to select between them, much like tabs in a web browser window.

It is possible to tell niri that an application's windows should always be opened in floating mode, and more. The [window rules](#) documentation has a full list of examples on setting window parameters, including size, decorations, even their opacity. It is also possible to set up what niri calls [named workspaces](#); these are workspaces that are always created, even if they have no active windows. One benefit of that feature is that niri can be configured to always place certain application windows in a named workspace, so all windows of a certain type are grouped on one workspace of their own.

This could be useful to, say, have all browser windows on one workspace, or all terminal windows.

The Super+O binding will invoke niri's new overview feature to see, as well as navigate between, all open windows and workspaces using arrow keys (or mouse or touchpad). The figure below shows part of niri's overview—unfortunately, its built-in screen capture utility does not have a mode that can capture the full overview across multiple displays.



Unlike desktop environments, niri does not have automatic power-saving features, nor does it lock the screen after a period of inactivity. I'd become accustomed to GNOME doing those things by default. On Fedora, [swaylock](#) is installed by default with niri for screen locking. The Mod+Shift+P shortcut will turn off displays; any key press will wake them up again. To exit niri, use Super+Shift+E and then press Enter when the confirmation dialog is displayed.

Configuring niri

Niri also lacks a graphical configuration application. Any settings changes need to be made by editing the plain-text configuration file, which uses the [KDL](#) format. It would probably be possible to use niri "out of the box" on Fedora without any additional configuration, but most people will want to tweak its settings at least a little bit.

By default, niri uses a system-wide configuration; there is no user configuration file set up. To get a starter configuration file, copy the `/usr/share/doc/niri/default-config.kdl` file included with niri to `~/.config/niri/config.kdl`. The default-config.kdl file has many of the possible configuration options sorted by sections, such as input, output, and binds, with useful comments.

Changes to the configuration generally take effect as soon as the configuration file is written. For example, niri's initial setup uses the Mod+T key binding to launch the [Alacritty](#) terminal. If a user wanted to change that to, say, [Ghostty](#), the configuration would look like this:

```
Mod+T hotkey-overlay-title="Open Ghostty" { spawn "ghostty"; }
```

The first field sets the key binding, Mod+T. The second, "hotkey-overlay-title", sets the title that is displayed in the "Important Hotkeys" dialog. The third, "{ spawn "ghostty"; }", specifies the command that should be run. Users are not limited to the keyboard when setting up bindings for niri; it also supports mouse clicks, mouse scroll-wheel actions, and touchpad actions. For instance, one can set a combination of Mod plus the mouse scroll wheel to move up or down between workspaces:

```
binds {  
  Mod+WheelScrollDown { focus-workspace-down; }  
  Mod+WheelScrollUp { focus-workspace-up; }  
}
```

Multi-display setups

The [documentation](#) on the niri wiki is excellent overall; the only real gap that I've noticed is for multi-display setups. They are [covered](#), but more examples would be beneficial. Niri will automatically detect and attempt to configure multiple monitors properly without the user needing to tweak the configuration. In my experience, it does fine at setting the proper resolution for displays, but it's still necessary to adjust the configuration for placement since niri has no way to know how the monitors are arranged physically.

Niri *does* handle multi-display setups with varying resolutions, rotations, and scaling well. It just takes some trial-and-error to get it right if one has a complicated setup. For example, I tested niri with a three-display configuration that included a laptop screen, high-resolution widescreen, and a smaller HD monitor turned sideways.

The project has a useful command ("niri msg outputs") to help with this. It displays the name of each display it detects, as well as information such as each one's available resolutions and refresh rates.

Xwayland

Out of the box, niri does not support X11 applications or Xwayland. Though Xwayland is a Wayland client, it [requires special support](#) from a Wayland compositor—work that Molodetskikh [does not plan](#) to do because, in his words, "X11 is very cursed". Niri can, however, work with [xwayland-satellite](#), which implements rootless Xwayland as a separate application, to run X11 applications. Right now, xwayland-satellite has to be configured separately, but it is expected [to be integrated with the next major release](#) of niri. For now, it can be enabled by adding a few lines to config.kdl. The first is the option to start the application when niri starts:

```
spawn-at-startup "xwayland-satellite"
```

It will also need an environment variable to set the display for xwayland-satellite:

```
environment {  
    DISPLAY ":0"  
}
```

So far, I have only tried a few X11 applications with xwayland-satellite, but it seems to do the job.

Pure imagination

Niri allows some fairly elaborate setups; the primary limitation on what one can do with niri is probably the person's imagination and the time they are willing to spend tweaking its configuration. It is pretty much the opposite of GNOME—niri is aimed at more advanced users, and it is meant for users to dig into the configuration and tweak things to their heart's content. There is a [showcase page on GitHub](#) where users have uploaded screenshots and linked to configuration files for their niri setups, which might be a good starting point for anyone looking for ideas or examples.

Users do not have to spend a great deal of time configuring niri, however; it took about two days before I was perfectly happy with my configuration. It did not require a lot of work to become productive with niri, just a bit of quality time with its documentation and a text editor.

As it stands right now, I will probably stick with niri. It seems stable, well-maintained, and the customization possibilities are almost endless. The scrollable-tiling model is, at least for my purposes, the ideal way to manage windows on large and small screens. It is easy to navigate and helps keep my hands on the keyboard where they belong.

to post comments

You do not have to use Waybar

[Link](#)

I have been using and loving Niri. The overview feature is simply incredible. You do not have to use it with Waybar though.

One of my machines, I use Niri with COSMIC panel and COSMIC terminal for example.

On another machine, use the XFCE panel and application suite, including using the XFCE application launcher instead of fuzzel. Again, it all works well (other than having to minimize the Xwayland video bridge on start-up).

It is quite fast on older hardware.

You do not have to use Waybar

[Link](#)

Inspiration

[Link](#) <https://desktopneo.com> <https://web.archive.org/web/20130120191904/http://10gui.com/>

Panning Virtual Desktop

[Link](#)

Panning Virtual Desktop

[Link](#)

Resource usage and start time

[Link](#)

wmii

[Link](#)

wmii

[Link](#)

Singing YaLTeR's praises

[Link](#)

papersway

[Link implemented scrollable tiling on top of Sway](#)

Niri with LXQt

[Link](#)

Produced by wallabag with tcpdf

Please open [an issue](#) if you have trouble with the display of this E-Book on your device.